# HOL 02 — Deploying a Hybrid Infrastructure for Researchers in AWS

**Student:** Marco Russo
**Email:** marco.russo@estudiants.urv.cat
**Date:** 2026-05-30
**Course:** High-Performance and Distributed Computing
**Lab:** HOL 02 — Hybrid Infrastructure (VPC + EC2 + S3 + Lambda)

---

## Architecture Overview

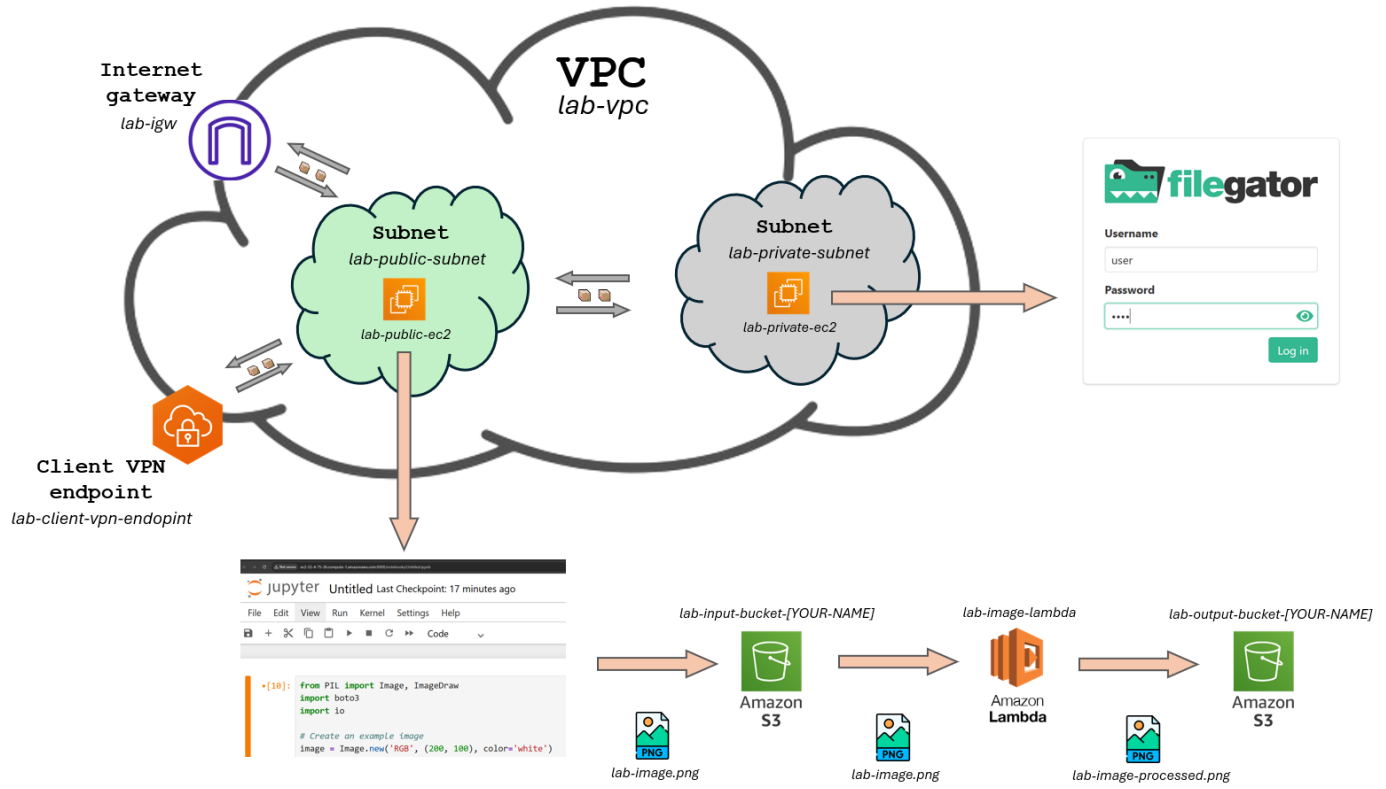
This lab deploys a hybrid cloud infrastructure on AWS consisting of:

- A **VPC** (`lab-vpc`) with a **public subnet** (hosting the Jupyter EC2) and a **private subnet** (hosting the file server EC2).
- An **Internet Gateway** attached to the public subnet for direct internet access.
- A **NAT Gateway** in the public subnet allowing the private EC2 to install software without being directly exposed.
- Two **S3 buckets**: one for incoming images (`lab-input-bucket-mrusso`) and one for processed results (`lab-output-bucket-mrusso`).
- A **Lambda function** triggered by S3 uploads that renames and moves images to the output bucket.
- A **Client VPN endpoint** (optional, from Session 6-7) to securely reach the private subnet from a local machine.

> **Deployment script used:** `deploy_hol02.sh` (automated all steps via AWS CLI).

---

## Achitecture Diagram




---

> **DISCLAIMER:** the code shown below has been made by bash script using AWS CLI. After testing within AWS Console, it was more easier to realise the same laboratory in other way and more efficient.

# Step 1 — VPC and Public Subnet

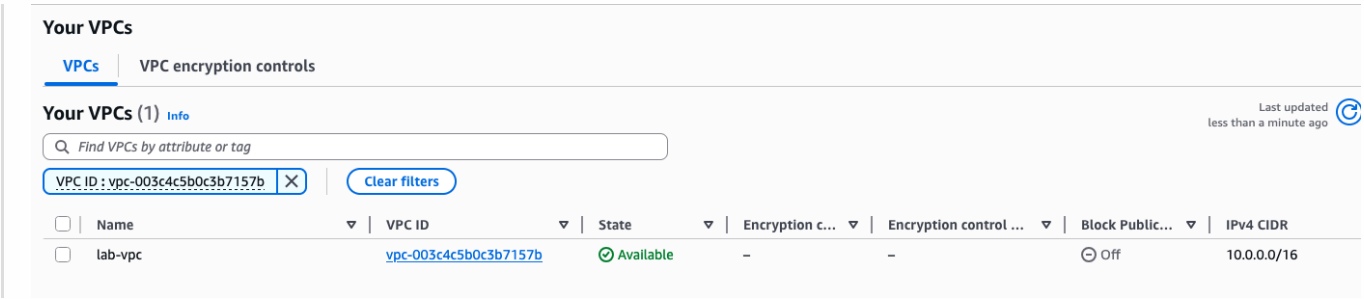
**Command run:**

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 --region us-east-1
aws ec2 create-subnet --cidr-block 10.0.1.0/24 --availability-zone us-east-1a
aws ec2 create-subnet --cidr-block 10.0.2.0/24 --availability-zone us-east-1b
```

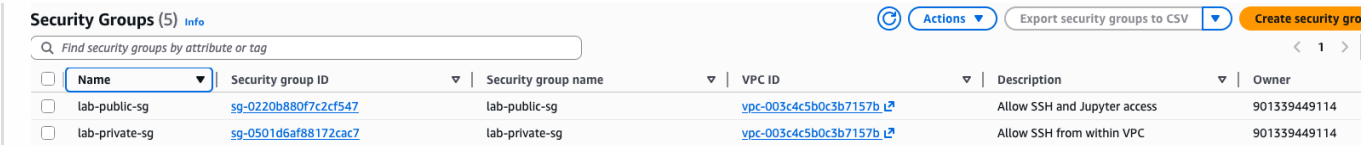
**Resources created:**

| Resource | Name | Value |
|---|---|---|
| VPC | `lab-vpc` | CIDR: `10.0.0.0/16` |
| Public Subnet | `lab-public-subnet` | CIDR: `10.0.1.0/24` |
| Private Subnet | `lab-private-subnet` | CIDR: `10.0.2.0/24` |

**Screenshot — VPC Console:**




**Screenshot — Subnets Console:**




---

# Step 2 — Internet Gateway

**Command run:**

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id <igw-id> --vpc-id <vpc-id>
```

**Resources created:**

| Resource | Name | Attached to |
|---|---|---|
| Internet Gateway | `lab-igw` | `lab-vpc` |

**Screenshot — Internet Gateway:**

---

# Step 3 — Route Tables

**Public Route Table (to Internet)**:

- *Route*: 0.0.0.0/0 → lab-igw
- *Association*: lab-public-subnet

**Private Route Table (to NAT)**:

- *Route*: 0.0.0.0/0 → lab-nat-gw
- *Association*: lab-private-subnet

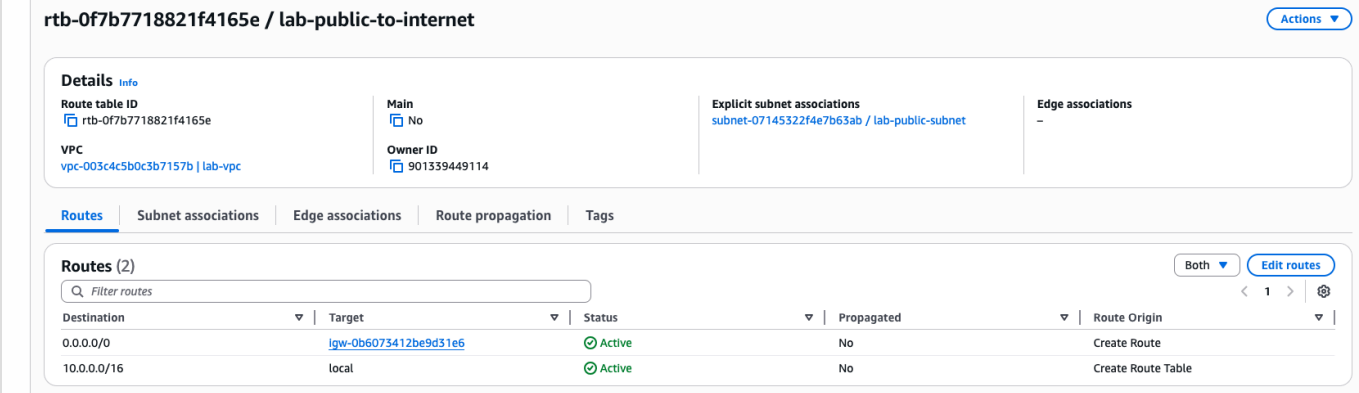
**Command run:**

```
aws ec2 create-route-table ...
aws ec2 create-route --destination-cidr-block 0.0.0.0/0 --gateway-id <igw-id>
aws ec2 associate-route-table --subnet-id <public-subnet-id> ...
```

**Routes configured:**

| Route Table | Destination | Target | Subnet |
|---|---|---|---|
| lab-public-to-internet | 0.0.0.0/0 | lab-igw | lab-public-subnet |
| lab-private-to-nat | 0.0.0.0/0 | lab-nat-gw | lab-private-subnet |

**Screenshot — Route Tables:**




---

# Step 4 — NAT Gateway

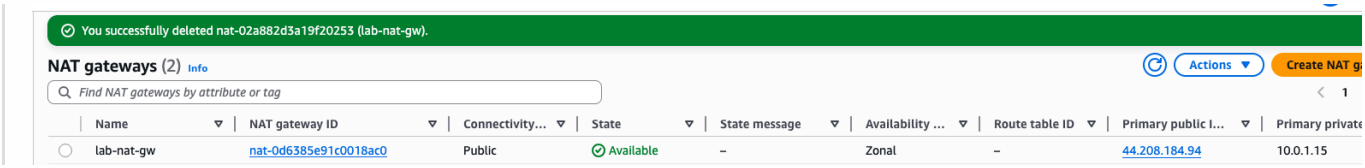
**Command run:**

```
aws ec2 allocate-address --domain vpc
aws ec2 create-nat-gateway --subnet-id <public-subnet-id> --allocation-id <eip-id>
aws ec2 create-vpc-endpoint --service-name com.amazonaws.us-east-1.s3 --vpc-id <vpc-id> --route-table-ids <private-rt-id>
```

**Resources created:**

| Resource | Name | Location |
|---|---|---|
| Elastic IP | — | Allocated to VPC |
| NAT Gateway | lab-nat-gw | lab-public-subnet |

**Screenshot — NAT Gateway:**




---

# Step 5 — Security Groups

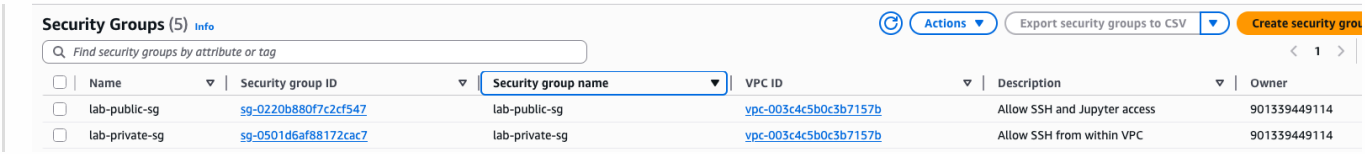
**Command run:**

```
aws ec2 create-security-group --group-name lab-public-sg ...
aws ec2 authorize-security-group-ingress --port 22 ...
aws ec2 authorize-security-group-ingress --port 8888 ...
```

**Rules configured:**

| Security Group | Protocol | Port | Source |
|---|---|---|---|
| `lab-public-sg` | TCP | 22 (SSH) | `0.0.0.0/0` |
| `lab-public-sg` | TCP | 8888 (Jupyter) | `0.0.0.0/0` |
| `lab-private-sg` | TCP | 22 (SSH) | `10.0.0.0/16` (VPC) |
| `lab-private-sg` | TCP | 80 (Filegator) | `10.0.0.0/16` (VPC) |

**Screenshot — Security Groups:**




---

# Step 6 — EC2 Instances

**Public EC2 (Jupyter server)**:

- *Name*: lab-public-ec2
- *Subnet*: lab-public-subnet
- *Public IP*: assigned automatically

**Private EC2 (file server):**

- *Name*: lab-private-ec2

- *Subnet*: lab-private-subnet
- *Public IP*: disabled

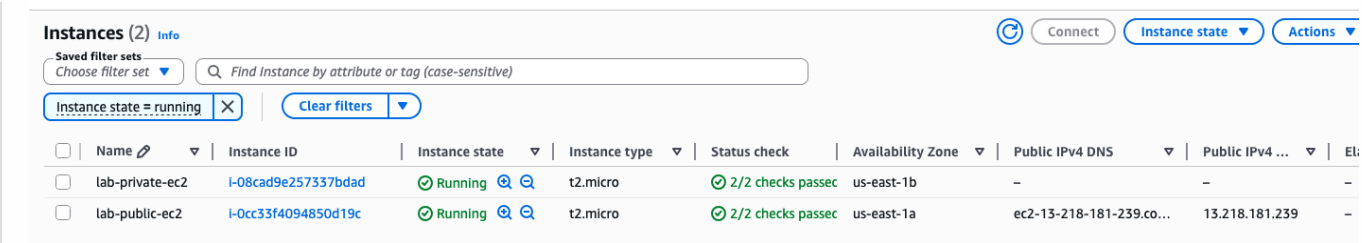
**Command run:**

```
aws ec2 run-instances --image-id <ami-id> --subnet-id <public-subnet-id> \
  --security-group-ids <public-sg-id> --associate-public-ip-address ...
aws ec2 run-instances --image-id <ami-id> --subnet-id <private-subnet-id> \
  --security-group-ids <private-sg-id> --no-associate-public-ip-address ...
```

**Resources created:**

| Instance | Name | Subnet | Public IP |
|---|---|---|---|
| `lab-public-ec2` | Public Jupyter server | `lab-public-subnet` | `<public-ip>` |
| `lab-private-ec2` | Private file server | `lab-private-subnet` | — (private only) |

**Screenshot — EC2 Instances running:**




---

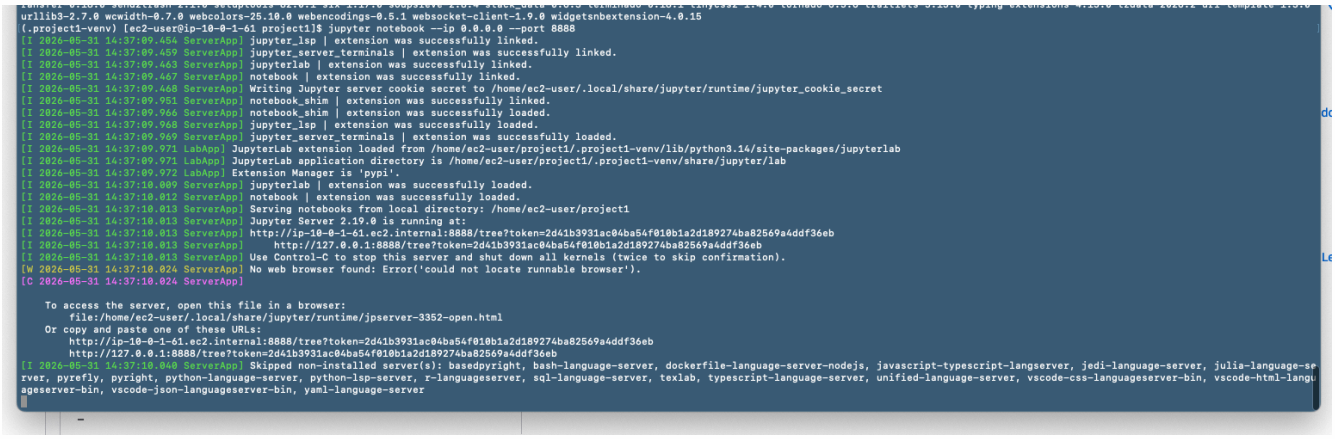
# Step 7 — Jupyter Notebook Setup on Public EC2

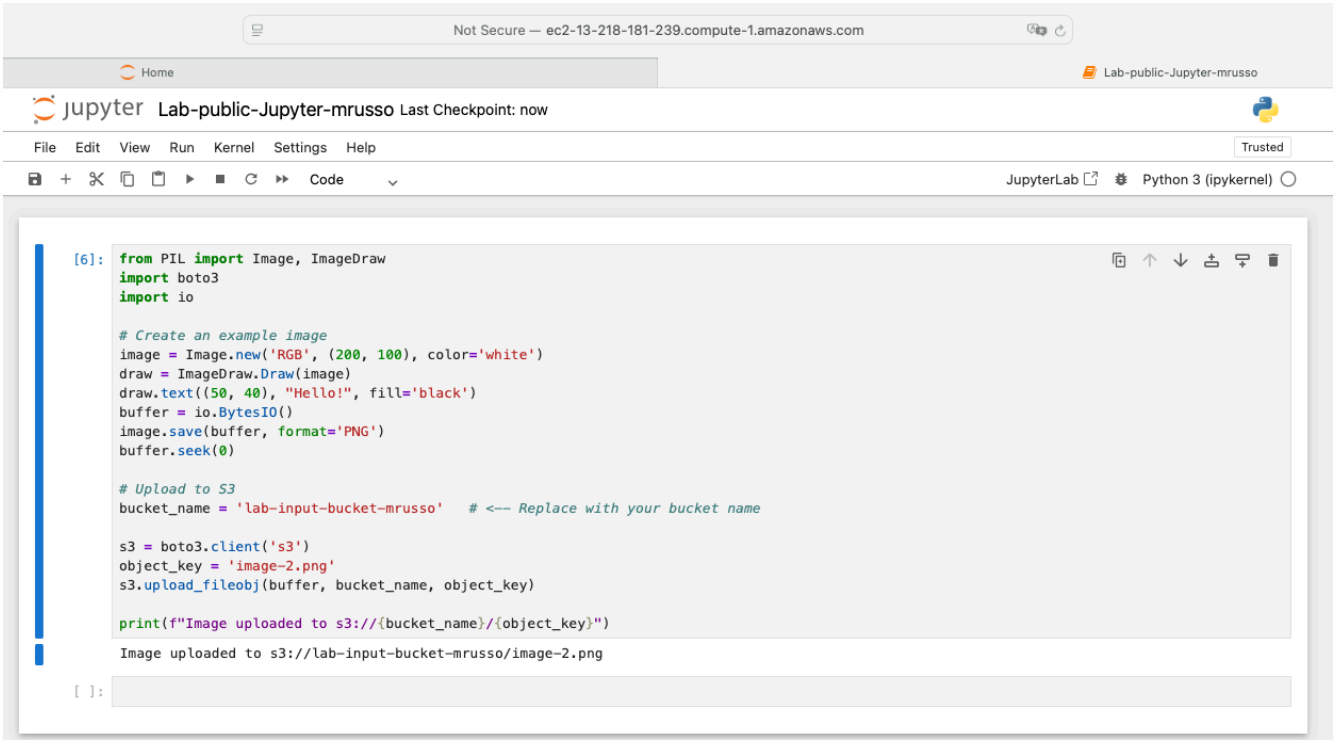
**Commands run on EC2 (via SSH):**

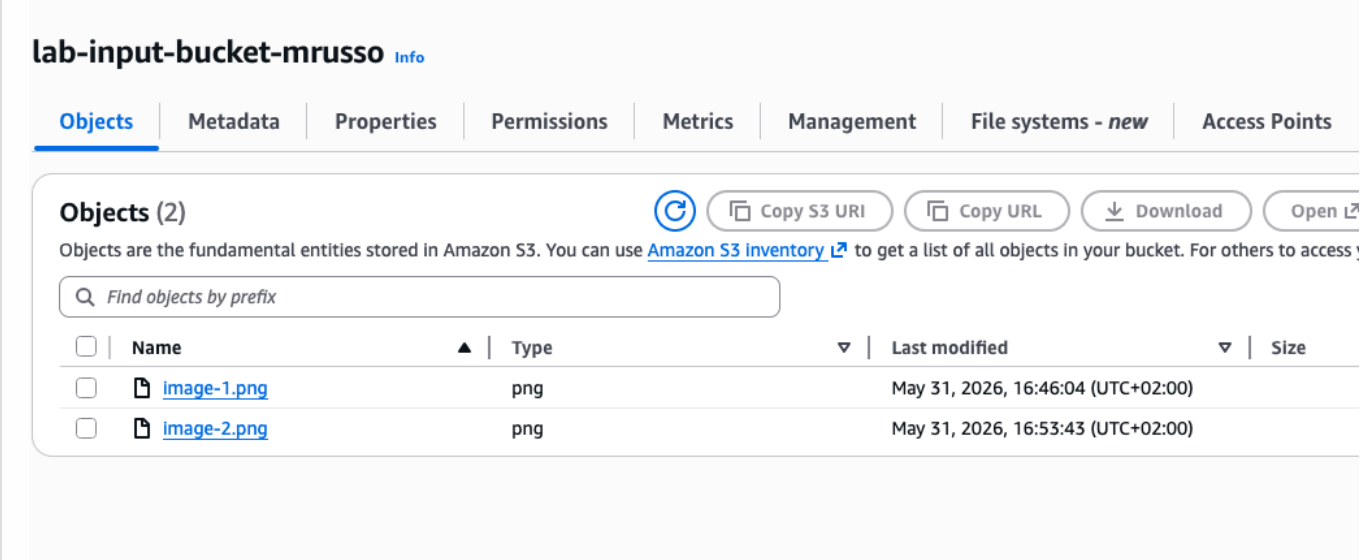
```
ssh -i aws-lab-keypair.pem ec2-user@<PUBLIC_IP>
sudo yum install -y python3 python3-pip
pip3 install boto3 jupyter pillow --user
aws configure
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser
```

**Screenshot — SSH connection to Public EC2:**


**Screenshot — Jupyter Notebook running in browser:**






# Step 8 — S3 Buckets

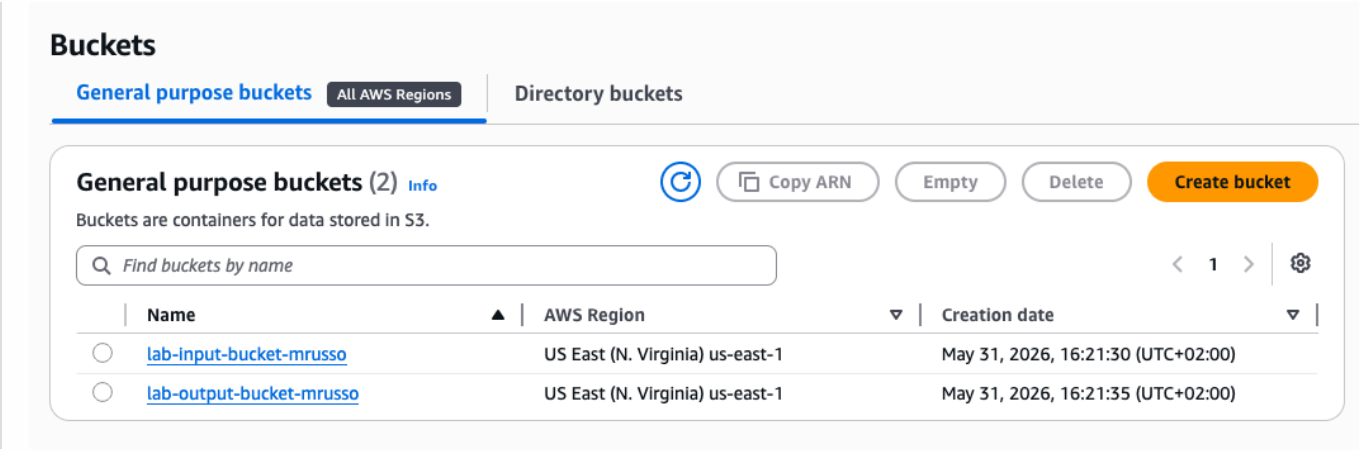
**Command run:**

```
aws s3api create-bucket --bucket lab-input-bucket-mrussorb --region us-east-1
aws s3api create-bucket --bucket lab-output-bucket-mrussorb --region us-east-1
```

**Resources created:**

| Bucket | Purpose |
|---|---|
| `lab-input-bucket-mrussorb` | Receives images uploaded from Jupyter |
| `lab-output-bucket-mrussorb` | Stores Lambda-processed images |

**Screenshot — S3 Buckets:**



# Step 9 — Lambda Function

**Lambda configuration**:

- *Name*: lab-lambda-function
- *Runtime*: Python 3.12
- *Handler*: lambda_function.lambda_handler
- *Trigger*: S3 event notification on lab-input-bucket-mrusso for *.png files in input/ prefix
- *IAM Role*: LabRole

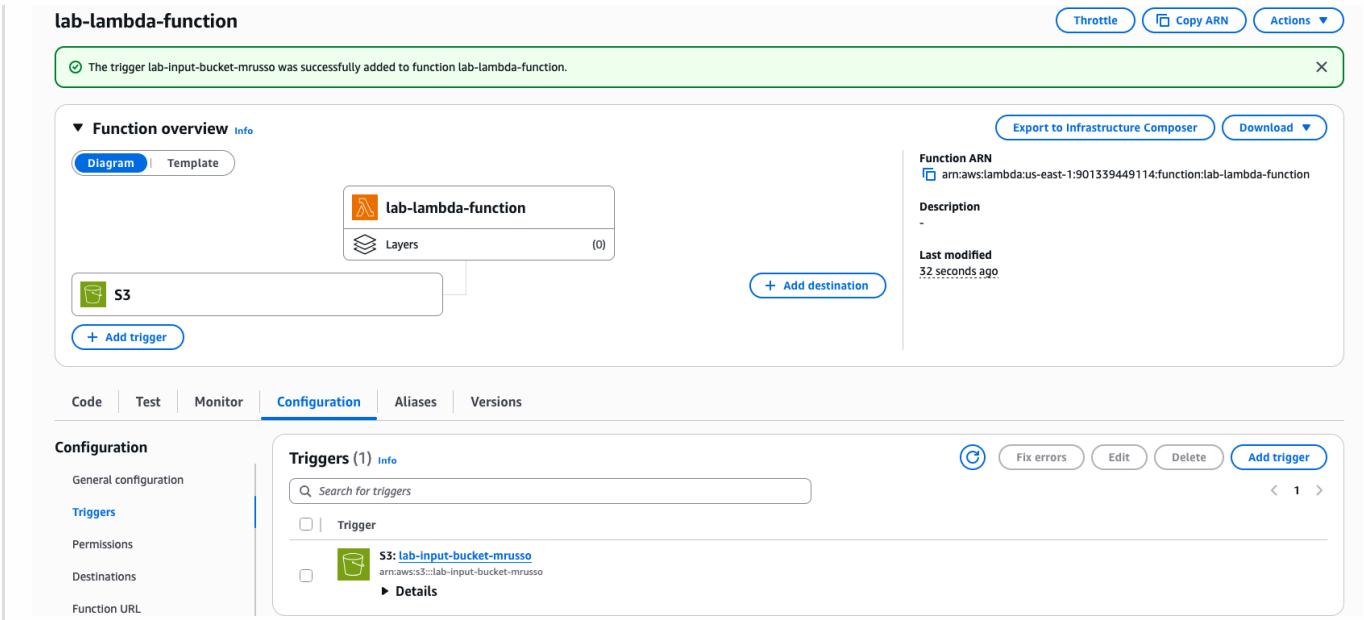
**Command run:**

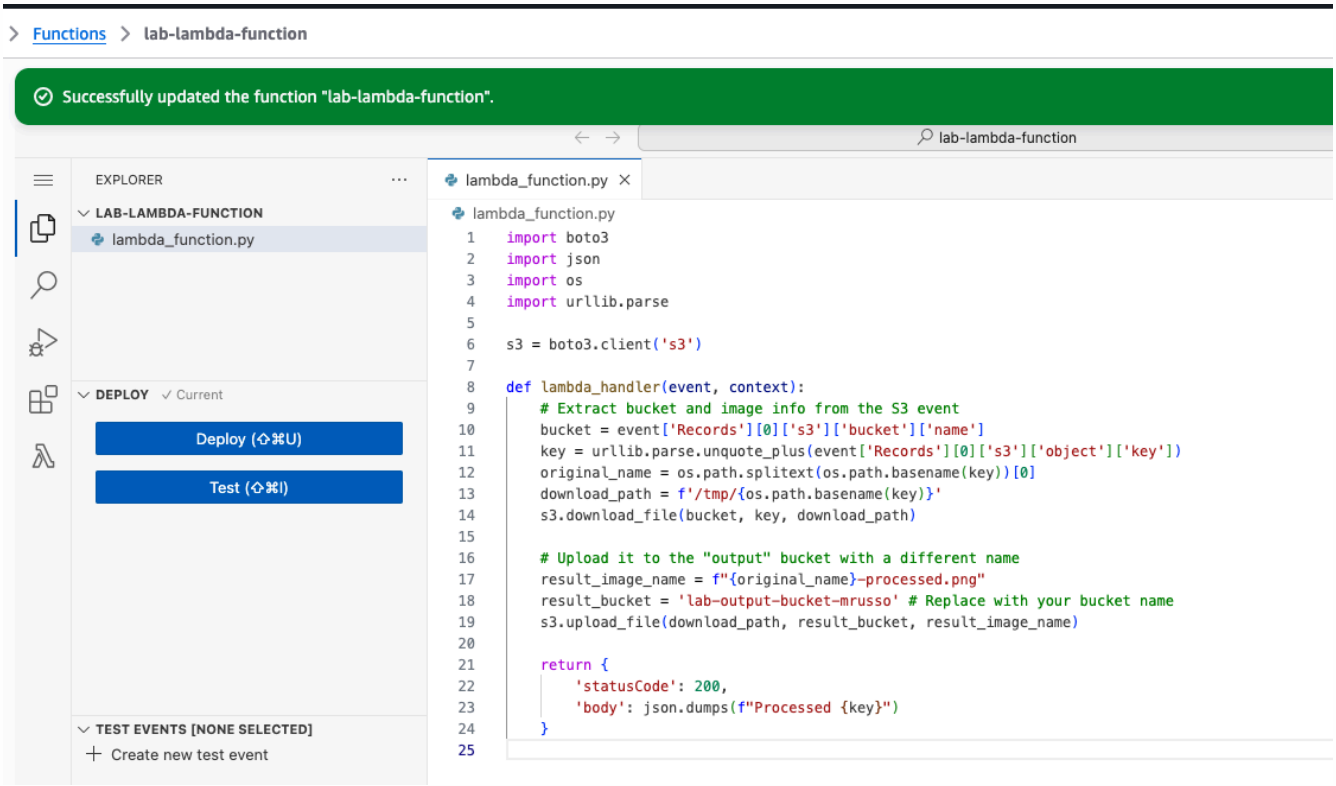
```
aws iam create-role --role-name lab-lambda-s3-role ...
aws lambda create-function --function-name lab-image-lambda \
  --runtime python3.12 --handler lambda_function.lambda_handler ...
aws s3api put-bucket-notification-configuration \
  --bucket lab-input-bucket-mrussorb ...
```

**Lambda code:**

```
import boto3, json, os, urllib.parse

s3 = boto3.client('s3')

def lambda_handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
    original_name = os.path.splitext(os.path.basename(key))[0]
    download_path = f'/tmp/{os.path.basename(key)}'

    s3.download_file(bucket, key, download_path)

    result_image_name = f"{original_name}-processed.png"
    result_bucket = 'lab-output-bucket-mrussorb'
    s3.upload_file(download_path, result_bucket, result_image_name)

    return {'statusCode': 200, 'body': json.dumps(f"Processed {key}")}
```

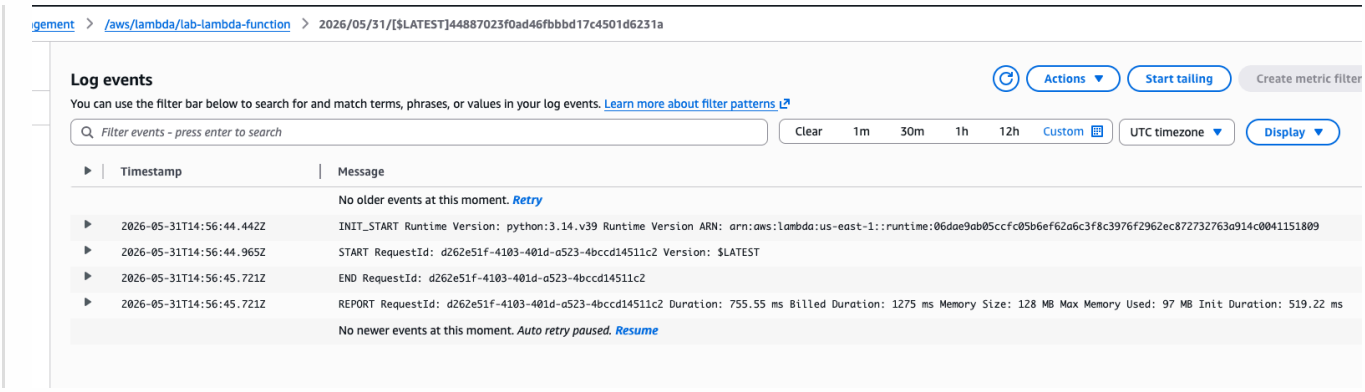
**Screenshot — Lambda Function created:**



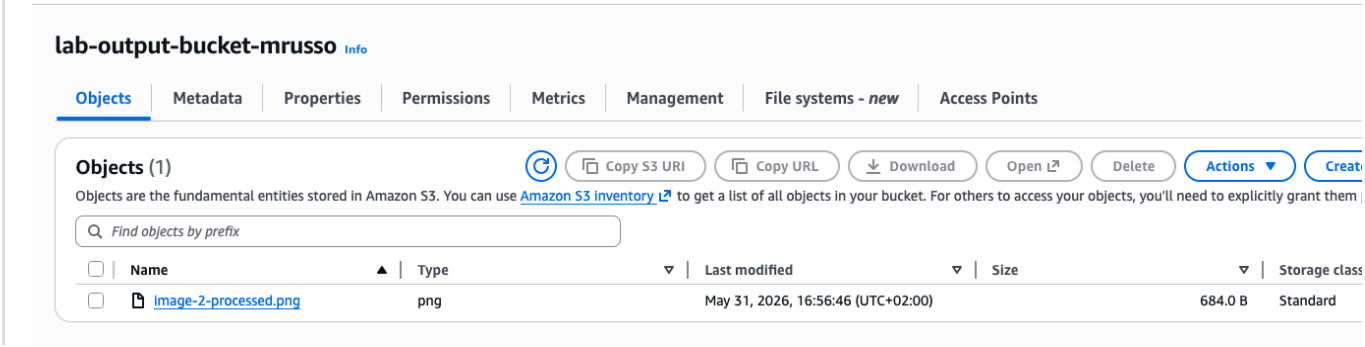




---

**Screenshot — Lambda execution logs (CloudWatch):**




**Screenshot — Output bucket with processed image:**




---

# Final Test

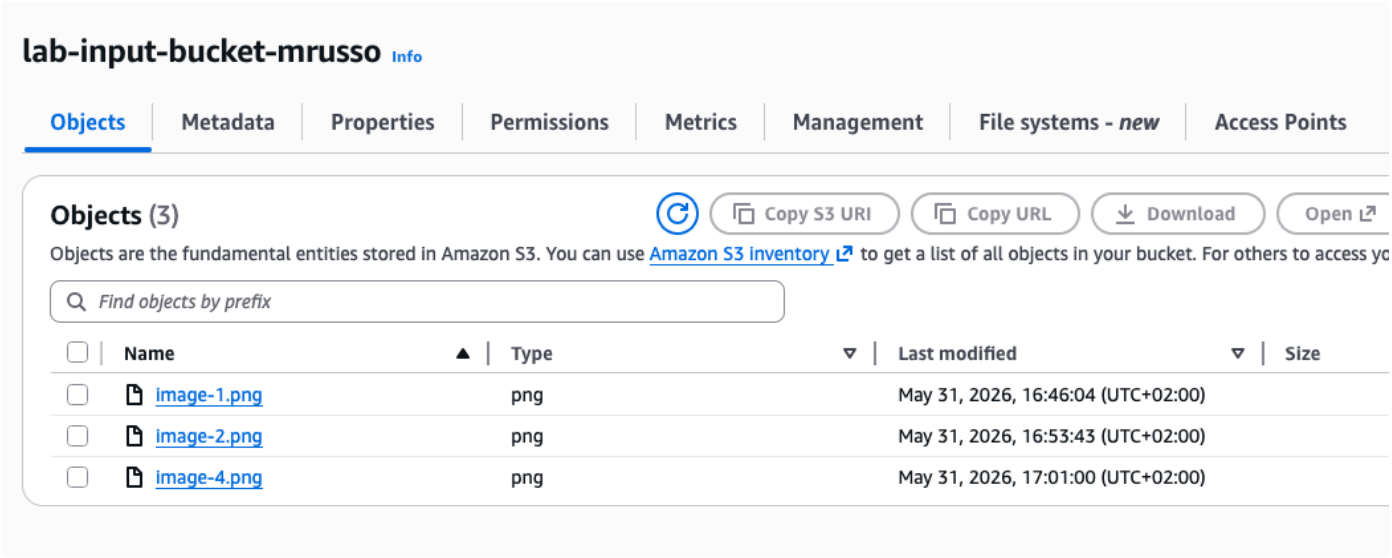
*Upload the new image in the Jupyter Notebook*

[7]:
```python
from PIL import Image, ImageDraw
import boto3
import io

# Create an example image
image = Image.new('RGB', (200, 100), color='white')
draw = ImageDraw.Draw(image)
draw.text((50, 40), "Hello!", fill='black')
buffer = io.BytesIO()
image.save(buffer, format='PNG')
buffer.seek(0)

# Upload to S3
bucket_name = 'lab-input-bucket-mrusso'   # <-- Replace with your bucket name

s3 = boto3.client('s3')
object_key = 'image-4.png'
s3.upload_fileobj(buffer, bucket_name, object_key)

print(f"Image uploaded to s3://{bucket_name}/{object_key}")
```

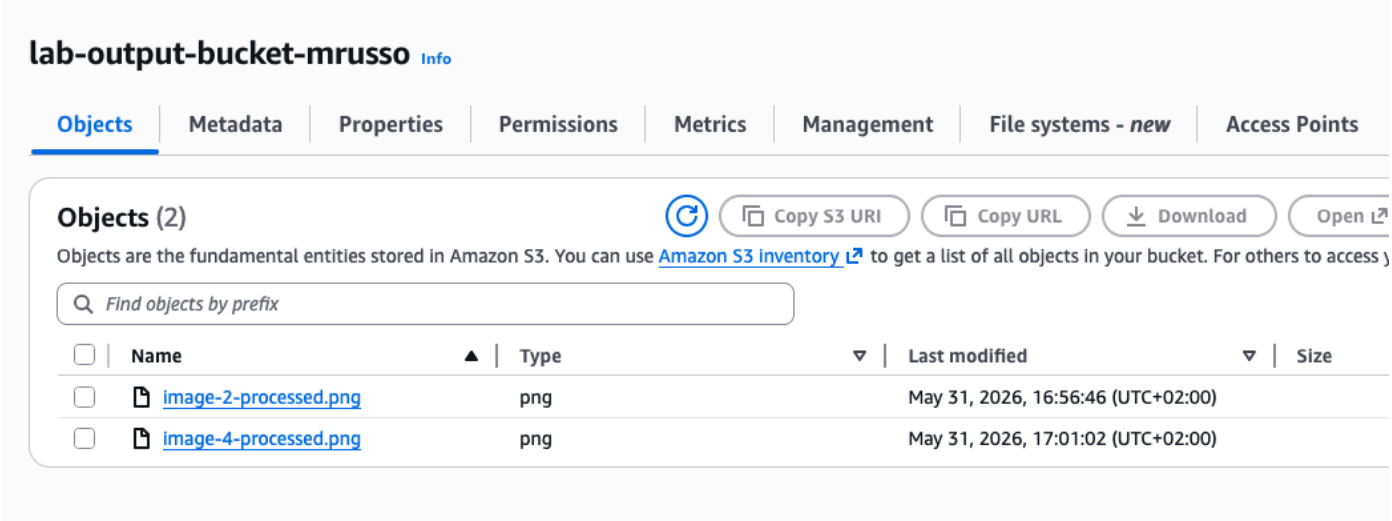
```
Image uploaded to s3://lab-input-bucket-mrusso/image-4.png
```

[ ]:

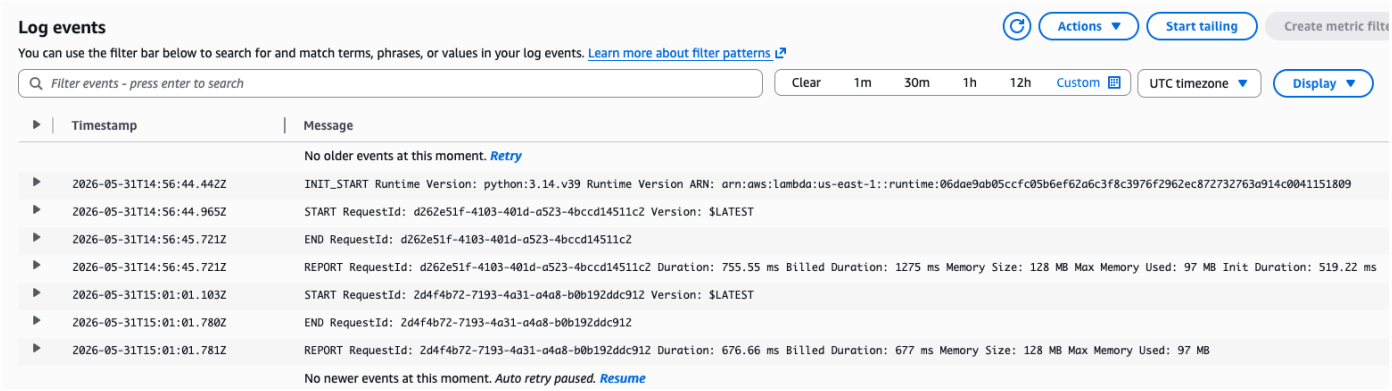
The image-4.png has been uploaded in the S3 Bucket `lab-input-bucket-mrusso`




The `lambda-function` has been triggered and process the new file to put on `lab-output-bucket-mrusso`




The cloudwatch event shows the results:




## NOTE:

The laboratory has been completed successfully, and to test EC2 instances are in running from 31st May 2026.

## DEPLOY_HOL02.SH

The following code has been used to deploy by AWS CLI as discussed earlier:

```
#!/bin/bash
# ================================================================================
# HOL 02 – Hybrid Infrastructure Deployment Script
# AWS Academy Learner Lab – CLI Only
# ================================================================================
# USAGE:
#   1. Set YOUR_NAME below (lowercase, no spaces)
#   2. Run: chmod +x deploy_hol02.sh && ./deploy_hol02.sh
```

```
#   3. Take screenshots of each step for your report
# ================================================================================

set -e  # Exit on any error

# ──────────────────────────────────────────────
# 0. CONFIGURATION — Edit this section only
# ──────────────────────────────────────────────
YOUR_NAME="mrusso"
AWS_REGION="us-east-1"
KEY_PAIR_NAME="aws-keypair-${YOUR_NAME}"

# Derived names (do not edit)
VPC_NAME="lab-vpc"
VPC_CIDR="10.0.0.0/16"
PUBLIC_SUBNET_NAME="lab-public-subnet"
PUBLIC_SUBNET_CIDR="10.0.1.0/24"
PRIVATE_SUBNET_NAME="lab-private-subnet"
PRIVATE_SUBNET_CIDR="10.0.2.0/24"
IGW_NAME="lab-igw"
PUBLIC_RT_NAME="lab-public-to-internet"
PRIVATE_RT_NAME="lab-private-to-nat"
PUBLIC_EC2_NAME="lab-public-ec2"
PRIVATE_EC2_NAME="lab-private-ec2"
PUBLIC_SG_NAME="lab-public-sg"
PRIVATE_SG_NAME="lab-private-sg"
INPUT_BUCKET="lab-input-bucket-${YOUR_NAME}"
OUTPUT_BUCKET="lab-output-bucket-${YOUR_NAME}"
LAYER_BUCKET="layers-bucket-${YOUR_NAME}"
LAMBDA_NAME="lab-lambda-function"
LAMBDA_ROLE_NAME="LabRole"
VPN_CONNECTION_NAME="lab-client-vpn-connection"


echo "============================================="
echo " HOL 02 Deployment — Region: $AWS_REGION"
echo " Student: $YOUR_NAME"
echo "============================================="
echo ""

# ──────────────────────────────────────────────
# 1. VPC
# ──────────────────────────────────────────────
echo ">>> [1/12] Creating VPC: $VPC_NAME ($VPC_CIDR)..."

VPC_ID=$(aws ec2 create-vpc \
  --cidr-block "$VPC_CIDR" \
  --region "$AWS_REGION" \
  --query 'Vpc.VpcId' \
  --output text)

aws ec2 create-tags \
```

```
  --resources "$VPC_ID" \
  --tags Key=Name,Value="$VPC_NAME" \
  --region "$AWS_REGION"

aws ec2 modify-vpc-attribute \
  --vpc-id "$VPC_ID" \
  --enable-dns-hostnames \
  --region "$AWS_REGION"

echo "    VPC created: $VPC_ID"

# ──────────────────────────────────────────────
# 2. SUBNETS
# ──────────────────────────────────────────────
echo ""
echo ">>> [2/12] Creating subnets..."

# Public Subnet
PUBLIC_SUBNET_ID=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" \
  --cidr-block "$PUBLIC_SUBNET_CIDR" \
  --availability-zone "${AWS_REGION}a" \
  --region "$AWS_REGION" \
  --query 'Subnet.SubnetId' \
  --output text)

aws ec2 create-tags \
  --resources "$PUBLIC_SUBNET_ID" \
  --tags Key=Name,Value="$PUBLIC_SUBNET_NAME" \
  --region "$AWS_REGION"

aws ec2 modify-subnet-attribute \
  --subnet-id "$PUBLIC_SUBNET_ID" \
  --map-public-ip-on-launch \
  --region "$AWS_REGION"

echo "    Public subnet created: $PUBLIC_SUBNET_ID"

# Private Subnet
PRIVATE_SUBNET_ID=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" \
  --cidr-block "$PRIVATE_SUBNET_CIDR" \
  --availability-zone "${AWS_REGION}b" \
  --region "$AWS_REGION" \
  --query 'Subnet.SubnetId' \
  --output text)

aws ec2 create-tags \
  --resources "$PRIVATE_SUBNET_ID" \
  --tags Key=Name,Value="$PRIVATE_SUBNET_NAME" \
  --region "$AWS_REGION"
```

```
echo "    Private subnet created: $PRIVATE_SUBNET_ID"

# ──────────────────────────────────────────────
# 3. INTERNET GATEWAY
# ──────────────────────────────────────────────
echo ""
echo ">>> [3/12] Creating & attaching Internet Gateway: $IGW_NAME..."

IGW_ID=$(aws ec2 create-internet-gateway \
  --region "$AWS_REGION" \
  --query 'InternetGateway.InternetGatewayId' \
  --output text)

aws ec2 create-tags \
  --resources "$IGW_ID" \
  --tags Key=Name,Value="$IGW_NAME" \
  --region "$AWS_REGION"

aws ec2 attach-internet-gateway \
  --internet-gateway-id "$IGW_ID" \
  --vpc-id "$VPC_ID" \
  --region "$AWS_REGION"

echo "    Internet Gateway created and attached: $IGW_ID"

# ──────────────────────────────────────────────
# 4. NAT GATEWAY (for Private Subnet)
# ──────────────────────────────────────────────
echo ""
echo ">>> [4/12] Creating NAT Gateway (Elastic IP + NAT in public subnet)..."

EIP_ALLOC=$(aws ec2 allocate-address \
  --domain vpc \
  --region "$AWS_REGION" \
  --query 'AllocationId' \
  --output text)

echo "    Elastic IP allocated: $EIP_ALLOC"

NAT_GW_ID=$(aws ec2 create-nat-gateway \
  --subnet-id "$PUBLIC_SUBNET_ID" \
  --allocation-id "$EIP_ALLOC" \
  --region "$AWS_REGION" \
  --query 'NatGateway.NatGatewayId' \
  --output text)

aws ec2 create-tags \
  --resources "$NAT_GW_ID" \
  --tags Key=Name,Value="lab-nat-gw" \
  --region "$AWS_REGION"

echo "    NAT Gateway created: $NAT_GW_ID"
```

```
echo "    Waiting for NAT Gateway to become available (≈60s)..."

aws ec2 wait nat-gateway-available \
  --nat-gateway-ids "$NAT_GW_ID" \
  --region "$AWS_REGION"

echo "    NAT Gateway is available."

# ──────────────────────────────────────────────
# 5. ROUTE TABLES
# ──────────────────────────────────────────────
echo ""
echo ">>> [5/12] Creating Route Tables..."

# Public Route Table
PUBLIC_RT_ID=$(aws ec2 create-route-table \
  --vpc-id "$VPC_ID" \
  --region "$AWS_REGION" \
  --query 'RouteTable.RouteTableId' \
  --output text)

aws ec2 create-tags \
  --resources "$PUBLIC_RT_ID" \
  --tags Key=Name,Value="$PUBLIC_RT_NAME" \
  --region "$AWS_REGION"

# Route: 0.0.0.0/0 -> IGW
aws ec2 create-route \
  --route-table-id "$PUBLIC_RT_ID" \
  --destination-cidr-block "0.0.0.0/0" \
  --gateway-id "$IGW_ID" \
  --region "$AWS_REGION" > /dev/null

# Associate with public subnet
aws ec2 associate-route-table \
  --route-table-id "$PUBLIC_RT_ID" \
  --subnet-id "$PUBLIC_SUBNET_ID" \
  --region "$AWS_REGION" > /dev/null

echo "    Public route table created: $PUBLIC_RT_ID (0.0.0.0/0 -> $IGW_ID)"

# Private Route Table
PRIVATE_RT_ID=$(aws ec2 create-route-table \
  --vpc-id "$VPC_ID" \
  --region "$AWS_REGION" \
  --query 'RouteTable.RouteTableId' \
  --output text)

aws ec2 create-tags \
  --resources "$PRIVATE_RT_ID" \
  --tags Key=Name,Value="$PRIVATE_RT_NAME" \
  --region "$AWS_REGION"
```

```
# Route: 0.0.0.0/0 -> NAT GW
aws ec2 create-route \
  --route-table-id "$PRIVATE_RT_ID" \
  --destination-cidr-block "0.0.0.0/0" \
  --nat-gateway-id "$NAT_GW_ID" \
  --region "$AWS_REGION" > /dev/null

# Associate with private subnet
aws ec2 associate-route-table \
  --route-table-id "$PRIVATE_RT_ID" \
  --subnet-id "$PRIVATE_SUBNET_ID" \
  --region "$AWS_REGION" > /dev/null

echo "    Private route table created: $PRIVATE_RT_ID (0.0.0.0/0 -> $NAT_GW_ID)"

# ──────────────────────────────────────────────
# 6. SECURITY GROUPS
# ──────────────────────────────────────────────
echo ""
echo ">>> [6/12] Creating Security Groups..."

# Public EC2 Security Group
PUBLIC_SG_ID=$(aws ec2 create-security-group \
  --group-name "$PUBLIC_SG_NAME" \
  --description "Allow SSH and Jupyter access" \
  --vpc-id "$VPC_ID" \
  --region "$AWS_REGION" \
  --query 'GroupId' \
  --output text)

aws ec2 create-tags \
  --resources "$PUBLIC_SG_ID" \
  --tags Key=Name,Value="$PUBLIC_SG_NAME" \
  --region "$AWS_REGION"

# SSH (22)
aws ec2 authorize-security-group-ingress \
  --group-id "$PUBLIC_SG_ID" \
  --protocol tcp \
  --port 22 \
  --cidr "0.0.0.0/0" \
  --region "$AWS_REGION" > /dev/null

# Jupyter (8888)
aws ec2 authorize-security-group-ingress \
  --group-id "$PUBLIC_SG_ID" \
  --protocol tcp \
  --port 8888 \
  --cidr "0.0.0.0/0" \
  --region "$AWS_REGION" > /dev/null
```

```
echo "    Public SG created: $PUBLIC_SG_ID (SSH:22, Jupyter:8888)"

# Private EC2 Security Group
PRIVATE_SG_ID=$(aws ec2 create-security-group \
  --group-name "$PRIVATE_SG_NAME" \
  --description "Allow SSH from within VPC" \
  --vpc-id "$VPC_ID" \
  --region "$AWS_REGION" \
  --query 'GroupId' \
  --output text)

aws ec2 create-tags \
  --resources "$PRIVATE_SG_ID" \
  --tags Key=Name,Value="$PRIVATE_SG_NAME" \
  --region "$AWS_REGION"

# SSH only from VPC CIDR
aws ec2 authorize-security-group-ingress \
  --group-id "$PRIVATE_SG_ID" \
  --protocol tcp \
  --port 22 \
  --cidr "$VPC_CIDR" \
  --region "$AWS_REGION" > /dev/null

# Filegator (port 80)
aws ec2 authorize-security-group-ingress \
  --group-id "$PRIVATE_SG_ID" \
  --protocol tcp \
  --port 80 \
  --cidr "$VPC_CIDR" \
  --region "$AWS_REGION" > /dev/null

echo "    Private SG created: $PRIVATE_SG_ID (SSH:22 from VPC, HTTP:80 from VPC)"

# ─────────────────────────────────────────────
# 7. KEY PAIR
# ─────────────────────────────────────────────
echo ""
echo ">>> [7/12] Checking Key Pair: $KEY_PAIR_NAME..."

EXISTING_KEY=$(aws ec2 describe-key-pairs \
  --key-names "$KEY_PAIR_NAME" \
  --region "$AWS_REGION" \
  --query 'KeyPairs[0].KeyName' \
  --output text 2>/dev/null || echo "NOT_FOUND")

if [ "$EXISTING_KEY" = "$KEY_PAIR_NAME" ]; then
  # Key pair already exists in AWS
  if [ -f "${KEY_PAIR_NAME}.pem" ]; then
    # .pem file also present locally — safe to reuse
    echo "    Key pair '$KEY_PAIR_NAME' already exists in AWS and .pem found locally.
Reusing it."
```

```
  else
    # Key exists in AWS but .pem is missing — cannot recover private key, must
recreate
    echo "    WARNING: Key pair '$KEY_PAIR_NAME' exists in AWS but .pem is missing
locally."
    echo "    Deleting remote key pair and recreating to obtain the .pem file..."
    aws ec2 delete-key-pair \
      --key-name "$KEY_PAIR_NAME" \
      --region "$AWS_REGION"
    aws ec2 create-key-pair \
      --key-name "$KEY_PAIR_NAME" \
      --region "$AWS_REGION" \
      --query 'KeyMaterial' \
      --output text > "${KEY_PAIR_NAME}.pem"
    chmod 400 "${KEY_PAIR_NAME}.pem"
    echo "    Key pair recreated and saved to: ${KEY_PAIR_NAME}.pem"
  fi
else
  # Key pair does not exist — create it fresh
  aws ec2 create-key-pair \
    --key-name "$KEY_PAIR_NAME" \
    --region "$AWS_REGION" \
    --query 'KeyMaterial' \
    --output text > "${KEY_PAIR_NAME}.pem"
  chmod 400 "${KEY_PAIR_NAME}.pem"
  echo "    Key pair created and saved to: ${KEY_PAIR_NAME}.pem (keep this file
safe!)"
fi

# ─────────────────────────────────────────────
# 8. EC2 INSTANCES
# ─────────────────────────────────────────────
echo ""
echo ">>> [8/12] Launching EC2 instances..."

# Get latest Amazon Linux 2 AMI
AMI_ID=$(aws ec2 describe-images \
  --owners amazon \
  --filters \
    "Name=name,Values=amzn2-ami-hvm-2.0.*-x86_64-gp2" \
    "Name=state,Values=available" \
  --query 'sort_by(Images, &CreationDate)[-1].ImageId' \
  --region "$AWS_REGION" \
  --output text)

echo "    Using AMI: $AMI_ID (Amazon Linux 2)"

# Public EC2
PUBLIC_EC2_ID=$(aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type "t2.micro" \
  --key-name "$KEY_PAIR_NAME" \
```

```
  --subnet-id "$PUBLIC_SUBNET_ID" \
  --security-group-ids "$PUBLIC_SG_ID" \
  --associate-public-ip-address \
  --region "$AWS_REGION" \
  --query 'Instances[0].InstanceId' \
  --output text)

aws ec2 create-tags \
  --resources "$PUBLIC_EC2_ID" \
  --tags Key=Name,Value="$PUBLIC_EC2_NAME" \
  --region "$AWS_REGION"

echo "    Public EC2 launched: $PUBLIC_EC2_ID"

# Private EC2
PRIVATE_EC2_ID=$(aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type "t2.micro" \
  --key-name "$KEY_PAIR_NAME" \
  --subnet-id "$PRIVATE_SUBNET_ID" \
  --security-group-ids "$PRIVATE_SG_ID" \
  --no-associate-public-ip-address \
  --region "$AWS_REGION" \
  --query 'Instances[0].InstanceId' \
  --output text)

aws ec2 create-tags \
  --resources "$PRIVATE_EC2_ID" \
  --tags Key=Name,Value="$PRIVATE_EC2_NAME" \
  --region "$AWS_REGION"

echo "    Private EC2 launched: $PRIVATE_EC2_ID"
echo "    Waiting for both instances to be running..."

aws ec2 wait instance-running \
  --instance-ids "$PUBLIC_EC2_ID" "$PRIVATE_EC2_ID" \
  --region "$AWS_REGION"

# Get public IP of public EC2
PUBLIC_EC2_IP=$(aws ec2 describe-instances \
  --instance-ids "$PUBLIC_EC2_ID" \
  --region "$AWS_REGION" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text)

echo "    Both instances running!"
echo "    Public EC2 IP: $PUBLIC_EC2_IP"

# ──────────────────────────────────────────────
# 9. S3 BUCKETS
# ──────────────────────────────────────────────
echo ""
```

```
echo ">>> [9/12] Creating S3 Buckets..."

# Input bucket
aws s3api create-bucket \
  --bucket "$INPUT_BUCKET" \
  --region "$AWS_REGION" > /dev/null

aws s3api put-bucket-tagging \
  --bucket "$INPUT_BUCKET" \
  --tagging 'TagSet=[{Key=Name,Value=lab-input-bucket}]'

echo "    Input bucket created: s3://$INPUT_BUCKET"

# Output bucket
aws s3api create-bucket \
  --bucket "$OUTPUT_BUCKET" \
  --region "$AWS_REGION" > /dev/null

aws s3api put-bucket-tagging \
  --bucket "$OUTPUT_BUCKET" \
  --tagging 'TagSet=[{Key=Name,Value=lab-output-bucket}]'

echo "    Output bucket created: s3://$OUTPUT_BUCKET"

# ─────────────────────────────────────────────
# 10. LAMBDA IAM ROLE
# ─────────────────────────────────────────────

echo ""
echo ">>> [10/12] Resolving existing Lambda IAM Role..."

LAMBDA_ROLE_ARN=$(aws iam get-role \
  --role-name "$LAMBDA_ROLE_NAME" \
  --query 'Role.Arn' \
  --output text 2>/dev/null || echo "")

if [ -z "$LAMBDA_ROLE_ARN" ] || [ "$LAMBDA_ROLE_ARN" = "None" ]; then
  echo "    ERROR: Could not find role '$LAMBDA_ROLE_NAME' in this account."
  echo "    Run: aws iam list-roles --query 'Roles[*].RoleName' --output table"
  echo "    Then set LAMBDA_ROLE_ARN manually at the top of this script."
  exit 1
fi

echo "    Using existing role: $LAMBDA_ROLE_ARN"
sleep 10


# ─────────────────────────────────────────────
# 11. LAMBDA FUNCTION
# ─────────────────────────────────────────────
echo ""
echo ">>> [11/12] Creating Lambda function: $LAMBDA_NAME..."
```

```
# Write the Lambda handler
cat > /tmp/lambda_function.py << PYEOF
import boto3
import cv2
import numpy as np
import json
import os
import urllib.parse

s3 = boto3.client('s3')

def lambda_handler(event, context):
    # Extract bucket and image info from the S3 event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
    original_name = os.path.splitext(os.path.basename(key))[0]

    download_path = f'/tmp/{os.path.basename(key)}'

    # Download and load the image from S3
    s3.download_file(bucket, key, download_path)
    image = cv2.imread(download_path)

    # Count the cells using contours
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    adaptive_thresh = cv2.adaptiveThreshold(
        gray_image, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY_INV, 65, 5
    )
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
    morph_image = cv2.morphologyEx(adaptive_thresh, cv2.MORPH_OPEN, kernel,
iterations=1)
    contours, _ = cv2.findContours(
        morph_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    cell_count = len(contours)

    # Draw contours on a copy of the image
    output_image = image.copy()
    cv2.drawContours(output_image, contours, -1, (0, 255, 0), 2)

    # Save the processed image and upload it to the "processed" bucket
    result_image_name = f"{original_name}-processed-{cell_count}-cells.png"
    result_image_path = f'/tmp/{result_image_name}'
    cv2.imwrite(result_image_path, output_image)
    result_bucket = '${OUTPUT_BUCKET}'
    s3.upload_file(result_image_path, result_bucket, result_image_name)

    return {
      'statusCode': 200,
      'body': json.dumps(f"Processed {key}, found {cell_count} cells.")
```

```
    }
PYEOF

# Zip the Lambda package
cd /tmp && zip lambda_package.zip lambda_function.py && cd -

# Create Lambda function
LAMBDA_ARN=$(aws lambda create-function \
  --function-name "$LAMBDA_NAME" \
  --runtime "python3.12" \
  --role "$LAMBDA_ROLE_ARN" \
  --handler "lambda_function.lambda_handler" \
  --zip-file "fileb:///tmp/lambda_package.zip" \
  --timeout 30 \
  --region "$AWS_REGION" \
  --query 'FunctionArn' \
  --output text)

echo "    Lambda function created: $LAMBDA_ARN"

# Allow S3 to invoke Lambda
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

aws lambda add-permission \
  --function-name "$LAMBDA_NAME" \
  --statement-id "s3-trigger" \
  --action "lambda:InvokeFunction" \
  --principal "s3.amazonaws.com" \
  --source-arn "arn:aws:s3:::${INPUT_BUCKET}" \
  --source-account "$ACCOUNT_ID" \
  --region "$AWS_REGION" > /dev/null

echo "    Waiting for Lambda permissions to propagate..."
sleep 5

# Add S3 event notification to trigger Lambda
cat > /tmp/s3-notification.json << NOTIFEOF
{
  "LambdaFunctionConfigurations": [
    {
      "LambdaFunctionArn": "${LAMBDA_ARN}",
      "Events": ["s3:ObjectCreated:*"],
      "Filter": {
        "Key": {
          "FilterRules": [
            { "Name": "Prefix", "Value": "input/" },
            { "Name": "Suffix", "Value": ".png" }
          ]
        }
      }
    }
  ]
```

```
}
NOTIFEOF

aws s3api put-bucket-notification-configuration \
  --bucket "$INPUT_BUCKET" \
  --notification-configuration file:///tmp/s3-notification.json

echo "    S3 trigger configured: $INPUT_BUCKET -> $LAMBDA_NAME"

# ─────────────────────────────────────────────
# 12. LAMBDA LAYER
# ─────────────────────────────────────────────

echo ">>> [12/13] Creating S3 Buckets..."

echo "Create and publish a Lambda layer with the dependencies"
mkdir -p /lambda_layer/python/lib/python3.14/site-packages/
cd lambda_layer

echo "create a virtual environment with uv and install the dependencies"
uv venv --seed --python 3.14 .lambda_venv
source .lambda_venv/bin/activate
pip install opencv-python-headless numpy -t python/lib/python3.14/site-packages

echo "zip the layer package"
zip -r opencv.zip python

echo "create and upload the layer to S3"
aws s3 mb s3://$LAYER_BUCKET --region $AWS_REGION
aws s3 cp opencv.zip s3://$LAYER_BUCKET/opencv.zip

echo "create the Lambda layer in AWS"
LAYER_ARN=$(aws lambda publish-layer-version \
  --layer-name "opencv-layer" \
  --content "S3Bucket=$LAYER_BUCKET,S3Key=opencv.zip" \
  --compatible-runtimes "python3.14" \

echo "    Lambda layer created: $LAYER_ARN"
echo "Publish Lambda layer to Lambda funtion"
aws lambda update-function-configuration \
  --function-name "$LAMBDA_NAME" \
  --layers "$LAYER_ARN" \
  --region "$AWS_REGION"

# ─────────────────────────────────────────────
# 13. CREATE A VPN CONNECTION
# ─────────────────────────────────────────────

echo ">>> [13/14] Setting up Client VPN connection..."

echo "Generate server and client certificates using Easy-RSA..."
git clone https://github.com/OpenVPN/easy-rsa.git
```

```
cd easy-rsa/easyrsa3

./easyrsa init-pki
./easyrsa build-ca nopass --batch
./easyrsa build-server-full server nopass --batch
./easyrsa --san=DNS:server build-client-full client1 nopass --batch
./easyrsa build-client-full client1.domain.tld nopass --batch

echo "Upload server certificate to ACM..."
cd easyrsa3
mkdir ~/custom_folder/
cp pki/ca.crt ~/custom_folder/
cp pki/issued/server.crt ~/custom_folder/
cp pki/private/server.key ~/custom_folder/
cp pki/issued/client1.domain.tld.crt ~/custom_folder/
cp pki/private/client1.domain.tld.key ~/custom_folder/
cd ~/custom_folder/

echo "Create ACM certificate for the server..."
SERVER_CERT_ARN=$(aws acm import-certificate \
  --certificate fileb://server.crt \
  --private-key fileb://server.key \
  --certificate-chain fileb://ca.crt \
  --region "$AWS_REGION" \

echo "    Server certificate imported to ACM: $SERVER_CERT_ARN"
aws acm import-certificate --certificate fileb://client1.domain.tld.crt \
--private-key fileb://client1.domain.tld.key \
--certificate-chain fileb://ca.crt \
--region "$AWS_REGION"

echo "Create Client VPN endpoint..."
VPN_ENDPOINT_ID=$(aws ec2 create-client-vpn-endpoint \
  --client-cidr-block "10.83.0.0/16" \
  --server-certificate-arn "$SERVER_CERT_ARN" \
  --authentication-options "Type=certificate,MutualAuthentication=
{ClientRootCertificateChainArn=$CLIENT_CERT_ARN}" \
  --connection-log-options "Enabled=true,CloudWatchLogGroup=client-vpn-
logs,CloudWatchLogStream=client-vpn-log-stream" \
  --dns-servers "1.1.1.1" "8.8.8.8" \
  --region "$AWS_REGION" \
  --tag-specifications "ResourceType=client-vpn-endpoint,Tags=
[{Key=Name,Value=$VPN_CONNECTION_NAME}]" \
  --query 'ClientVpnEndpointId' \
  --output text)

echo "    Client VPN endpoint created: $VPN_ENDPOINT_ID"

echo "Associating Client VPN endpoint with public subnet ($PUBLIC_SUBNET_ID)..."

ASSOCIATION_ID=$(aws ec2 associate-client-vpn-target-network \
  --client-vpn-endpoint-id "$VPN_ENDPOINT_ID" \
```

```
  --subnet-id "$PUBLIC_SUBNET_ID" \
  --region "$AWS_REGION" \
  --query 'AssociationId' \
  --output text)

echo "    Target network association initiated. Association ID: $ASSOCIATION_ID"
echo "    Note: The association process takes a few minutes to transition from
'associating' to 'associated'."

echo "Adding authorization rule to allow VPN clients access to the VPC..."
aws ec2 authorize-client-vpn-ingress \
  --client-vpn-endpoint-id "$VPN_ENDPOINT_ID" \
  --target-network-cidr "$VPC_CIDR" \
  --authorize-all-groups \
  --region "$AWS_REGION"

echo "Checking VPN endpoint status..."
while true; do
  STATUS=$(aws ec2 describe-client-vpn-endpoints \
    --client-vpn-endpoint-ids "$VPN_ENDPOINT_ID" \
    --region "$AWS_REGION" \
    --query 'ClientVpnEndpoints[0].Status.Code' \
    --output text)
  echo "    Current VPN endpoint status: $STATUS"
  if [ "$STATUS" = "available" ]; then
    echo "    VPN endpoint is now available!"
    break
  fi
  sleep 10
done

# ------------------------------------------------
# 12. SUMMARY
# ------------------------------------------------
echo ""
echo "================================================"
echo " DEPLOYMENT COMPLETE — Resource Summary"
echo "================================================"
echo ""
echo "  VPC:              $VPC_ID ($VPC_CIDR)"
echo "  Public Subnet:    $PUBLIC_SUBNET_ID ($PUBLIC_SUBNET_CIDR)"
echo "  Private Subnet:   $PRIVATE_SUBNET_ID ($PRIVATE_SUBNET_CIDR)"
echo "  Internet GW:      $IGW_ID"
echo "  NAT Gateway:      $NAT_GW_ID"
echo "  Public Route RT:  $PUBLIC_RT_ID"
echo "  Private Route RT: $PRIVATE_RT_ID"
echo "  Public SG:        $PUBLIC_SG_ID"
echo "  Private SG:       $PRIVATE_SG_ID"
echo "  Public EC2:       $PUBLIC_EC2_ID (IP: $PUBLIC_EC2_IP)"
echo "  Private EC2:      $PRIVATE_EC2_ID"
echo "  Input Bucket:     s3://$INPUT_BUCKET"
echo "  Output Bucket:    s3://$OUTPUT_BUCKET"
```

```
echo "  Lambda:           $LAMBDA_NAME"
echo "  Key Pair file:    ${KEY_PAIR_NAME}.pem"
echo ""
echo "=========================================="
echo " NEXT STEPS — Run on Public EC2"
echo "=========================================="
echo ""
echo "  SSH into public EC2:"
echo "  ssh -i ${KEY_PAIR_NAME}.pem ec2-user@$PUBLIC_EC2_IP"
echo ""
echo "  Then inside the EC2, run setup_jupyter.sh"
echo "  (see companion script)"
echo "=========================================="
```